Dual Channel Pressure Monitoring System

Project Report

Prepared by- Md. Jaied Hasan

Design Approach

I built on the ESP32 microcontroller platform to meet all project requirements. The design leverages ESP32's integrated WiFi capability for external event logging via a web-based dashboard.

- Architecture Overview:
  - Two analog pressure sensors (approximated as potentiometers) connected to ADC pins providing 12-bit resolution
  - A 16x2 LCD display showing active channel, pressure reading, threshold, and mode
  - Three pushbuttons used for: MODE switch, UNIT switch, CHANNEL switch
  - Dedicated digital outputs for each channel (blue LED, red LED) and buzzer for alarm
  - A dashboard providing remote monitoring and threshold configuration
  - NVS (Non-Volatile Storage) using ESP32 Preferences library for power-cycle persistence

The button interface employs a modal design with two operating modes. Normal mode allows channel switching and unit toggling between kPa and mmHg. Threshold mode enables threshold adjustment for the active channel.

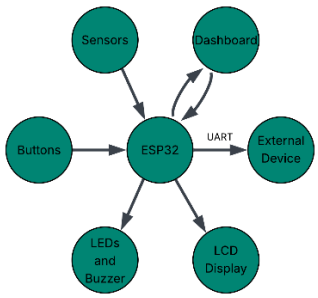

Fig 1. System architecture

Implementation Process and Results

Here are the details about how I have implemented and meet the project requirements-

1. **Real time pressure display, unit toggle and channel switch**

   The ESP32's 12-bit ADC reads raw sensor values (0-4095) and maps them to 0-200 kPa range. Here I have assumed both the sensors have range of 0-200 kPa. All internal calculations maintain kPa as the base unit. Conversion to mmHg performed

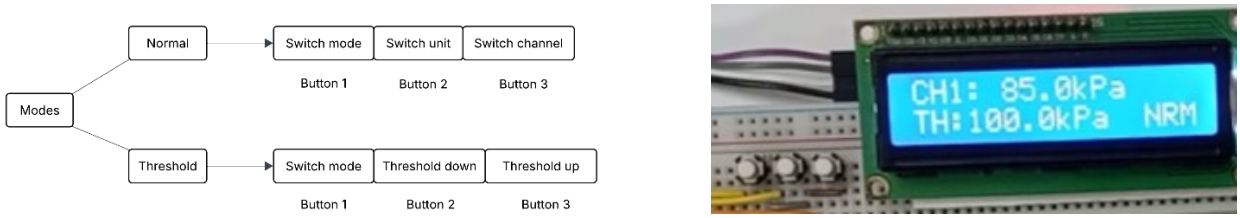

Fig. 2 Button map and LCD display

   only for display purposes. Three buttons and an LCD display are used to implement the first task. Buttons are used to switch mode, switch channel, switch unit, increase and decrease threshold. LCD display is refreshed in every 300ms.

2. **Alarm threshold update**

   The user can update the alarm threshold value for any channel in any unit using the button interface. He just needs to select channel and press threshold up/down button.

Disclaimer: Helps from AI tools were taken for refinement, formatting and grammatical corrections.

3. **Surviving power cycle**

   The ESP32 Preferences library provides key-value storage in flash memory. Four parameters are saved: threshold values for both channels, active channel selection, and display unit preference. The system loads these values during initialization and writes them immediately after any user modification. Testing confirmed that settings survive complete power loss.

4. **Threshold breach alarm**

   There are 2 LEDs, each dedicated for one channel, and a buzzer. In case of threshold breach, the corresponding LED lights up and the buzzer sounds.

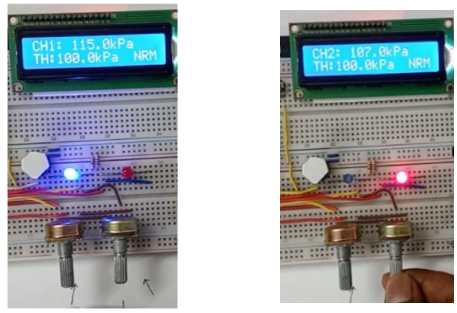

Fig 3. Threshold breach of channel-1(left) and channel-2(right)

5. **Event logging**

   Timestamped event logs are generated when channels cross thresholds or return to normal. Logging can happen in both wireless and wired setup. In wired condition, the ESP32 sends timestamp and event details to another device using UART protocol. ESP fetches local time from wifi router. In wireless mode, ESP sends timestamped logs to web dashboard and user can view and download logs as CSV file.

Fig 4. Timestamped log in UART device(left) and web dashboard(right)

6. **Web Dashboard**

   The ESP32 runs a lightweight HTTP server exposing three REST endpoints: /data (JSON sensor readings and events), /set (threshold configuration), and /download (CSV export). The dashboard HTML file can be run from any location. The dashboard polls the ESP32 every second for live updates of both channel pressure readings and current threshold values. Units can be toggled in dashboard too. It also provides a visual alarm for any threshold breach. User can set thresholds through the dashboard. CSV file containing timestamped logs can be downloaded from the dashboard.

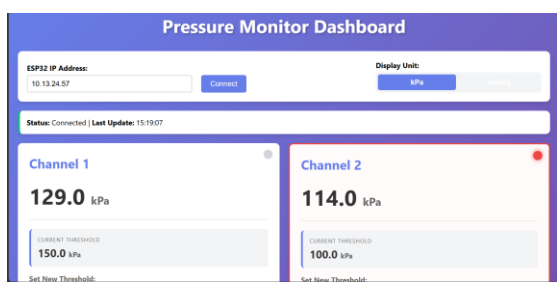

Fig 3. Web Dashboard

Known Limitations

- WiFi Dependency: Event logging through dashboard requires active WiFi connection. Local button functionality and alarm outputs remain operational during network outages, but remote logging is suspended.
- Time Synchronization: Accurate timestamps depend on successful NTP synchronization at startup. If NTP fails, events are logged with "NO_TIME" marker until connection is established.

Disclaimer: Helps from AI tools were taken for refinement, formatting and grammatical corrections.